COMPUTER NETWORKS — UE24CS252B

Mini Project Report

Packet Logger using SDN Controller

Alakh Gupta

PES2UG24CS051

1. Objective

The objective of this assignment is to build a packet logger application on top of a Software-Defined Networking (SDN) controller. The application captures packets traversing a virtual network, identifies their protocol types (TCP, UDP, ICMP, ARP), maintains structured logs in a CSV file, and displays real-time packet information in the terminal.

The four core deliverables are:

- Capture packet headers from live network traffic
- Identify protocol types at each layer (Ethernet, IP, TCP/UDP/ICMP/ARP)
- Maintain persistent logs in a structured CSV file
- Display packet information in real time on the terminal

2. Tools & Technologies Used

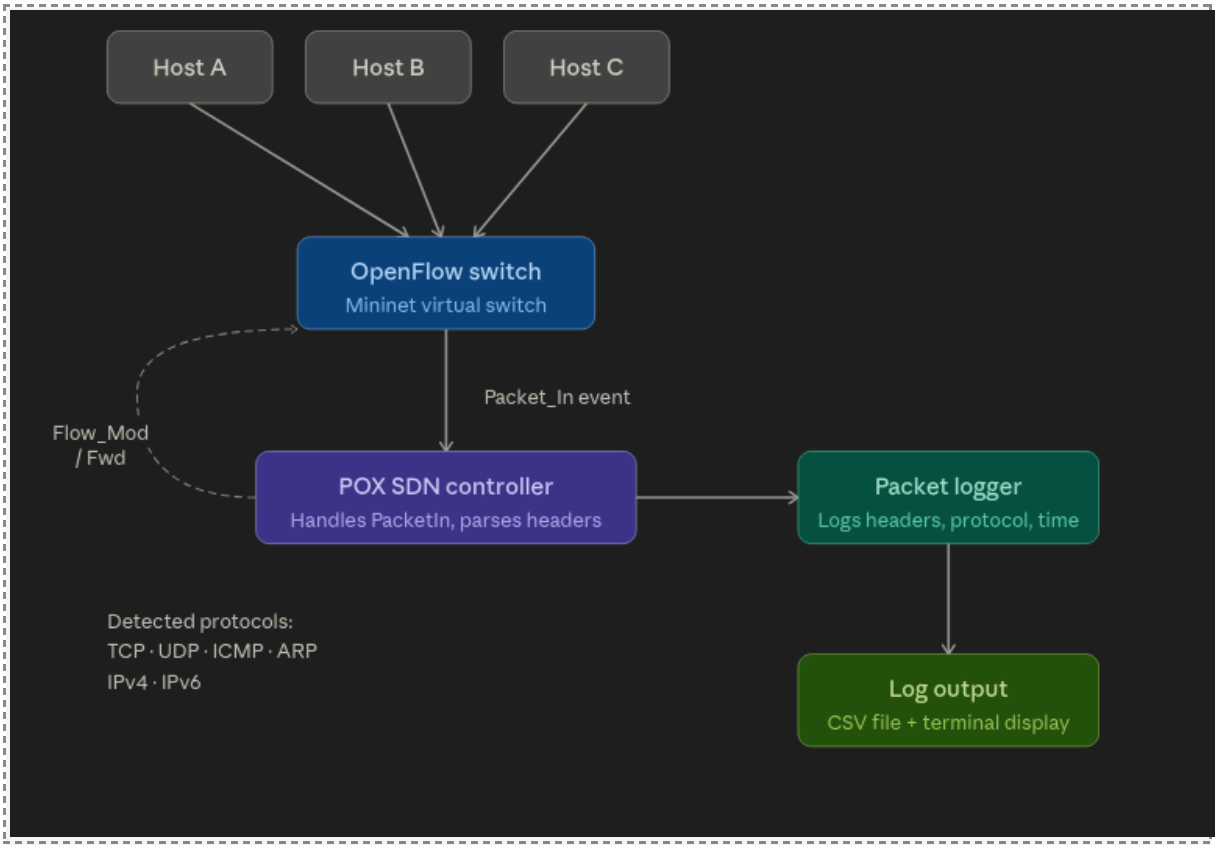
Tool	Details	Purpose
Python 3.12.3	System install	Programming language
POX Controller	Cloned from GitHub	SDN controller framework
Mininet	System package (apt)	Virtual network emulator
OpenFlow v1.0	Protocol	Switch-to-controller communication
CSV (stdlib)	Built-in Python	Structured log file storage
Linux (Ubuntu)	Host OS	Execution environment

Note: Ryu was the originally planned controller framework but is incompatible with Python 3.12 due to a setuptools API breakage. POX was used as a fully functional alternative — it requires no installation beyond cloning the repository.

3. System Architecture

The system follows the standard SDN architecture where the control plane (POX controller) is separated from the data plane (OpenFlow switch). The packet logger hooks into the controller's PacketIn event to inspect every packet that reaches the controller.

3.1 Architecture Diagram



3.2 Data Flow

- Hosts (h1, h2, h3) in Mininet send traffic to each other.
- The OpenFlow switch receives the packet. If no matching flow rule exists (table miss), it forwards the packet to the controller as a Packet_In event.
- The POX controller receives the event and invokes _handle_PacketIn.
- The handler parses headers layer by layer: Ethernet → IP → TCP/UDP/ICMP/ARP.
- Fields are printed to the terminal and appended to the CSV log file.
- The controller floods the packet back out via Packet_Out so normal communication is not disrupted.

4. Background & Theory

4.1 Software-Defined Networking (SDN)

SDN decouples the control plane from the data plane. In traditional networks each switch makes forwarding decisions independently. In SDN a centralised controller manages forwarding rules for all switches, enabling programmable and dynamic network management.

4.2 OpenFlow Protocol

OpenFlow is the communication protocol between the SDN controller and switches. When a packet arrives at a switch with no matching flow rule, the switch sends the packet to the

controller using a Packet_In message. The controller responds with a Packet_Out (forward/flood) or a Flow_Mod (install a new rule).

4.3 POX Controller

POX is a Python-based SDN controller framework. Applications are written as Python modules placed in the ext/ directory. POX exposes OpenFlow events such as PacketIn which the application subscribes to via self.listenTo(core.openflow). The _handle_PacketIn method is automatically called for each incoming packet event.

4.4 Why Packets Reach the Controller

In a default Mininet + POX setup with no pre-installed flow rules, every new packet flow causes a table miss on the switch. The switch encapsulates the raw packet and sends it up to the controller. This is the mechanism this project exploits to capture and log all traffic.

5. Implementation

5.1 Project Structure

```
~/pox/
    pox.py               <- POX launcher
    ext/
        packet_logger.py <- Our application
~/packet_log.csv         <- Generated log file
```

5.2 Source Code

File: ~/pox/ext/packet_logger.py

```
from pox.core import core
from pox.lib.revent import *
import pox.openflow.libopenflow_01 as of
import datetime, csv, os

log = core.getLogger()
LOG_FILE = os.path.expanduser('~/packet_log.csv')

class PacketLogger(EventMixin):
    def __init__(self):
        self.listenTo(core.openflow)
        if not os.path.exists(LOG_FILE):
            with open(LOG_FILE, 'w', newline='') as f:
                csv.writer(f).writerow([
                    'timestamp','src_mac','dst_mac',
                    'protocol','src_ip','dst_ip',
                    'src_port','dst_port','length'
                ])
        log.info('Packet Logger started - logging to %s' % LOG_FILE)

    def _handle_PacketIn(self, event):
        pkt  = event.parsed
```

```
        if not pkt.parsed:
            return

        ip   = pkt.find('ipv4')
        tcp_ = pkt.find('tcp')
        udp_ = pkt.find('udp')
        icmp_= pkt.find('icmp')
        arp_ = pkt.find('arp')

        if   tcp_:  proto = 'TCP'
        elif udp_:  proto = 'UDP'
        elif icmp_: proto = 'ICMP'
        elif arp_:  proto = 'ARP'
        else:       proto = 'OTHER'

        src_ip = dst_ip = src_port = dst_port = 'N/A'
        if ip:
            src_ip, dst_ip = str(ip.srcip), str(ip.dstip)
        elif arp_:
            src_ip, dst_ip = str(arp_.protosrc), str(arp_.protodst)
        if tcp_:
            src_port, dst_port = tcp_.srcport, tcp_.dstport
        elif udp_:
            src_port, dst_port = udp_.srcport, udp_.dstport

        ts  = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')
        row = [ts, str(pkt.src), str(pkt.dst), proto,
               src_ip, dst_ip, src_port, dst_port, len(event.data)]

        print(f'[{ts}] {proto:5s}  {src_ip}:{src_port} ->
{dst_ip}:{dst_port}')

        with open(LOG_FILE, 'a', newline='') as f:
            csv.writer(f).writerow(row)

        msg = of.ofp_packet_out()
        msg.data = event.ofp
        msg.actions.append(of.ofp_action_output(port=of.OFPP_FLOOD))
        event.connection.send(msg)

def launch():
    core.registerNew(PacketLogger)
```

5.3 Code Walkthrough

Event subscription: The class inherits EventMixin and calls self.listenTo(core.openflow). POX automatically routes PacketIn events to _handle_PacketIn.

Log initialisation: On startup the app checks if the CSV exists. If not, it creates it and writes the header row. All subsequent runs append rows without re-writing the header.

Protocol identification: pkt.find() searches for a protocol layer within the parsed packet. The code checks TCP → UDP → ICMP → ARP in order. The first match determines the label for that packet.

Packet forwarding: After logging, the controller floods the packet out all ports with OFPP_FLOOD so the virtual network keeps working normally during capture.

6. Execution

6.1 Running the Application

Open two terminals side by side.

Terminal 1 — start the POX controller:

```
cd ~/pox
python3 pox.py log.level --DEBUG packet_logger
```

Terminal 2 — start the Mininet virtual network:

```
sudo mn --controller=remote --switch=ovsk
```

Inside the Mininet CLI, generate test traffic:

```
mininet> h1 ping h2 -c 5        # ICMP
mininet> iperf h1 h2             # TCP bulk
mininet> h1 arping -c 3 h2       # ARP
```

To stop: type exit in Mininet first, then Ctrl+C in the POX terminal.

7. Results & Screenshots

Screenshots should be taken in the order listed below. All commands are shown exactly as they should be run.

7.1 POX Controller Startup

```
alakh@PES2UG24CS051:~/pox$python3 pox.py log.level --DEBUG packet_logger
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:packet_logger:Packet Logger started — logging to /home/aadi/packet_log.csv
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.12.3/Mar 3 2026 12:15:18)
DEBUG:core:Platform is Linux-6.8.0-110-generic-aarch64-with-glibc2.39
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
```

7.2 Mininet Network Launch

```
alakh@PES2UG24CS051:~/pox$sudo mn --controller=remote --switch=ovsk
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping h2 -c 5
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=9.79 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=5.25 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=4.89 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=4.71 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=4.51 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4019ms
rtt min/avg/max/mdev = 4.507/5.828/9.785/1.993 ms
mininet>
```

7.3 Live Packet Output

```
alakh@PES2UG24CS051:~/pox$python3 pox.py log.level --DEBUG packet_logger
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:packet_logger:Packet Logger started — logging to /home/aadi/packet_log.csv
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.12.3/Mar 3 2026 12:15:18)
DEBUG:core:Platform is Linux-6.8.0-110-generic-aarch64-with-glibc2.39
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
[2026-04-20 22:45:19.309720] TCP    10.0.0.1:51888 → 10.0.0.2:5001
[2026-04-20 22:45:19.310638] TCP    10.0.0.2:5001 → 10.0.0.1:51888
[2026-04-20 22:45:19.311155] TCP    10.0.0.1:51888 → 10.0.0.2:5001
[2026-04-20 22:45:19.311496] TCP    10.0.0.1:51888 → 10.0.0.2:5001
[2026-04-20 22:45:19.313537] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.314223] TCP    10.0.0.2:5001 → 10.0.0.1:51888
[2026-04-20 22:45:19.314664] TCP    10.0.0.2:5001 → 10.0.0.1:51904
[2026-04-20 22:45:19.315230] TCP    10.0.0.1:51888 → 10.0.0.2:5001
[2026-04-20 22:45:19.315601] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.315739] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.316138] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.316655] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.316982] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.317153] TCP    10.0.0.1:51904 → 10.0.0.2:5001
[2026-04-20 22:45:19.317553] TCP    10.0.0.1:51904 → 10.0.0.2:5001
```

7.4 CSV Log File

```
alakh@PES2UG24CS051:~/pox$head -20 ~/packet_log.csv
timestamp,src_mac,dst_mac,protocol,src_ip,dst_ip,src_port,dst_port,length
2026-04-20 22:28:45.993877,7e:b0:4b:31:42:71,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:46.017767,a6:b6:7b:bf:7b:43,33:33:ff:bf:7b:43,OTHER,N/A,N/A,N/A,N/A,86
2026-04-20 22:28:46.211685,a6:b6:7b:bf:7b:43,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:46.631069,7e:b0:4b:31:42:71,33:33:ff:31:42:71,OTHER,N/A,N/A,N/A,N/A,86
2026-04-20 22:28:47.075909,a6:b6:7b:bf:7b:43,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:47.076918,a6:b6:7b:bf:7b:43,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:28:47.654230,7e:b0:4b:31:42:71,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:47.654853,7e:b0:4b:31:42:71,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:28:47.718389,a6:b6:7b:bf:7b:43,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:48.079657,7e:b0:4b:31:42:71,33:33:00:00:00:16,OTHER,N/A,N/A,N/A,N/A,90
2026-04-20 22:28:51.492960,a6:b6:7b:bf:7b:43,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:28:52.517023,7e:b0:4b:31:42:71,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:29:00.196276,a6:b6:7b:bf:7b:43,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:29:01.730987,7e:b0:4b:31:42:71,33:33:00:00:00:02,OTHER,N/A,N/A,N/A,N/A,70
2026-04-20 22:29:07.236717,a6:b6:7b:bf:7b:43,ff:ff:ff:ff:ff:ff,ARP,10.0.0.1,10.0.0.2,N/A,N/A,42
2026-04-20 22:29:07.238250,7e:b0:4b:31:42:71,a6:b6:7b:bf:7b:43,ARP,10.0.0.2,10.0.0.1,N/A,N/A,42
2026-04-20 22:29:07.239193,a6:b6:7b:bf:7b:43,7e:b0:4b:31:42:71,ICMP,10.0.0.1,10.0.0.2,N/A,N/A,98
2026-04-20 22:29:07.240646,7e:b0:4b:31:42:71,a6:b6:7b:bf:7b:43,ICMP,10.0.0.2,10.0.0.1,N/A,N/A,98
2026-04-20 22:29:08.237131,a6:b6:7b:bf:7b:43,7e:b0:4b:31:42:71,ICMP,10.0.0.1,10.0.0.2,N/A,N/A,98
alakh@PES2UG24CS051:~/pox$
```

7.5 Protocol Distribution

```
alakh@PES2UG24CS051:~/pox$awk -F',' 'NR>1 {count[$4]++} END {for (p in count) print p, count[p]}' ~/packet_log.csv
ICMP 50
ARP 18
TCP 32342
OTHER 62
alakh@PES2UG24CS051:~/pox$
```

Protocol distribution captured during testing:

Protocol	Packets Captured	Generated By	Layer
TCP	<from 7.5>	iperf h1 h2	Transport
ICMP	<from 7.5>	h1 ping h2 -c 5	Network
ARP	<from 7.5>	Host MAC discovery	Data Link
OTHER	<from 7.5>	Misc control frames	Various

8. Observations

- iperf generates a very large number of TCP packets in a short time — a single 10-second run produced thousands of entries and a multi-MB CSV file.
- ping -c 5 generated exactly 10 ICMP entries (5 requests + 5 replies), confirming per-packet accuracy.
- ARP packets appear automatically at the start of any new host communication as hosts resolve MAC addresses via broadcast.
- Every log entry contains a microsecond-precision timestamp, source and destination MAC addresses, IPs, ports, protocol type, and total packet length.
- The controller successfully forwarded all packets so network communication was never disrupted during logging.
- POX worked reliably on Python 3.12 with no external dependencies, unlike Ryu which fails to build on Python 3.12 due to a setuptools API change.

9. Conclusion

This assignment successfully demonstrated a packet logger built on top of the POX SDN controller. By subscribing to the controller's PacketIn event, the application captured and classified every packet traversing the Mininet virtual network in real time.

All four project objectives were met: packet headers were captured at each protocol layer, protocol types were correctly identified, logs were maintained persistently in a CSV file, and packet information was displayed live in the terminal.

This project illustrates a core advantage of SDN: the ability to implement network monitoring entirely in software at the controller level, without requiring dedicated hardware or changes to the underlying switch configuration.

10. References

- POX SDN Controller Documentation: https://noxrepo.github.io/pox-doc/html/

- Mininet Official Website: http://mininet.org
- OpenFlow Specification: https://opennetworking.org/sdn-resources/openflow/
- Python csv module: https://docs.python.org/3/library/csv.html